Alternative Design Approaches for Match-3 Game

CURRENT APPROACH: Functional Immutable State

Pros: Thread-safe, testable, predictable **Cons:** Memory allocations, GC pressure **Best For:** Clarity, maintainability, distributed systems

---

ALTERNATIVE 1: Event-Driven Architecture (Event Sourcing)

Design Overview

Instead of storing current state, store all events that happened and replay them.

```java
```

```java
// Event hierarchy
sealed interface GameEvent permits BrickSpawned, BrickMoved, BrickPlaced, MatchesCleared {}
record BrickSpawned(Brick brick, Instant timestamp) implements GameEvent {}
record BrickMoved(Direction direction, Instant timestamp) implements GameEvent {}
record BrickPlaced(List<Block> blocks, Instant timestamp) implements GameEvent {}
record MatchesCleared(Set<Position> positions, Instant timestamp) implements GameEvent {}

// Event Store
class EventStore {
    private final List<GameEvent> events = new ArrayList<>();

    public void append(GameEvent event) {
        events.add(event);
    }

    public GameState reconstruct() {
        return events.stream()
            .reduce(GameState.empty(),
                (state, event) -> state.apply(event),
                (s1, s2) -> s2);
    }
}

// GameState applies events
class GameState {
    public GameState apply(GameEvent event) {
        return switch(event) {
            case BrickSpawned(var brick, var time) -> withBrick(brick);
            case BrickMoved(var dir, var time) -> moveBrick(dir);
            case BrickPlaced(var blocks, var time) -> placeBlocks(blocks);
            case MatchesCleared(var pos, var time) -> clearPositions(pos);
        };
    }
}
```

Advantages

- ✅ **Complete History**: Every action is recorded
- ✅ **Time Travel**: Can replay to any point
- ✅ **Debugging**: See exact sequence of events
- ✅ **Audit Trail**: Know what happened when
- ✅ **Undo/Redo**: Trivial to implement
- ✅ **Analytics**: Rich data for analysis

Disadvantages

❌ **Memory Growth**: Events accumulate
❌ **Complexity**: More moving parts
❌ **Performance**: Reconstruction can be slow
❌ **Learning Curve**: Less familiar pattern

When to Use

- Need complete audit trail
- Undo/redo is critical
- Building multiplayer with replay
- Need to debug complex state transitions
- Temporal queries ("what was state at 3pm?")

ALTERNATIVE 2: Mutable Object-Oriented (Traditional OOP)

Design Overview

Classic mutable objects with encapsulation.

```java
```

```java
class Field {
    private char[][] grid;
    private int width, height;

    public void placeBlock(int row, int col, char symbol) {
        grid[row][col] = symbol;
    }

    public void clearMatches() {
        // Directly mutate grid
        for (Position pos : findMatches()) {
            grid[pos.row()][pos.col()] = ' ';
        }
    }
}

class Brick {
    private Position position;
    private Orientation orientation;
    private List<Character> symbols;

    public void moveLeft() {
        position = new Position(position.row(), position.col() - 1);
    }

    public void moveDown() {
        position = new Position(position.row() + 1, position.col());
    }
}

class Game {
    private Field field;
    private Brick activeBrick;
    private Queue<Brick> brickQueue;

    public void tick() {
        processInput();
        activeBrick.moveDown();

        if (!field.canPlace(activeBrick)) {
            field.place(activeBrick);
            field.clearMatches();
            activeBrick = brickQueue.poll();
        }
```

```
    }
}
```

Advantages

✅ **Performance**: No object allocations per operation
✅ **Memory Efficient**: Single copy of state
✅ **Familiar**: Most developers know this pattern
✅ **Simple**: Straightforward to understand
✅ **Direct**: Modify data in place

Disadvantages

❌ **Threading**: Not thread-safe without locks
❌ **Testing**: Harder to test (need setup/teardown)
❌ **Debugging**: State changes are hidden
❌ **Bugs**: Mutation can cause unexpected side effects
❌ **Undo**: Complex to implement

When to Use

- Single-threaded application
- Performance is critical bottleneck
- Team familiar with traditional OOP
- Simple requirements (no undo, no multiplayer)
- Embedded systems with limited memory

ALTERNATIVE 3: Entity Component System (ECS)

Design Overview

Separate data (components) from behavior (systems).

```java
```

```java
// Components (pure data)
record PositionComponent(int row, int col) {}
record SymbolComponent(char symbol) {}
record VelocityComponent(int deltaRow, int deltaCol) {}
record BrickComponent(int brickId) {}

// Entity is just an ID
class Entity {
    private static int nextId = 0;
    private final int id;

    public Entity() { this.id = nextId++; }
    public int getId() { return id; }
}

// ECS World
class World {
    private Map<Entity, PositionComponent> positions = new HashMap<>();
    private Map<Entity, SymbolComponent> symbols = new HashMap<>();
    private Map<Entity, VelocityComponent> velocities = new HashMap<>();

    public void addComponent(Entity entity, Object component) {
        if (component instanceof PositionComponent pos) positions.put(entity, pos);
        if (component instanceof SymbolComponent sym) symbols.put(entity, sym);
        // ...
    }
}

// Systems (pure behavior)
class MovementSystem {
    public void update(World world) {
        world.getEntitiesWith(PositionComponent.class, VelocityComponent.class)
            .forEach(entity -> {
                PositionComponent pos = world.get(entity, PositionComponent.class);
                VelocityComponent vel = world.get(entity, VelocityComponent.class);

                world.update(entity, new PositionComponent(
                    pos.row() + vel.deltaRow(),
                    pos.col() + vel.deltaCol()
                ));
            });
    }
}

class MatchDetectionSystem {
    public void update(World world) {
```

```
        // Find entities with matching symbols in line
        // Remove them from world
    }
}

class RenderSystem {
    public void update(World world) {
        // Query all entities with Position and Symbol
        // Render to screen
    }
}

class Game {
    private World world = new World();
    private List<System> systems = List.of(
        new InputSystem(),
        new MovementSystem(),
        new CollisionSystem(),
        new MatchDetectionSystem(),
        new RenderSystem()
    );

    public void tick() {
        systems.forEach(system -> system.update(world));
    }
}
```

Advantages

✅ **Composition**: Flexible entity composition
✅ **Performance**: Cache-friendly data layout
✅ **Scalability**: Easy to parallelize systems
✅ **Decoupling**: Systems don't know about each other
✅ **Data-Oriented**: Optimized for modern CPUs
✅ **Dynamic**: Add/remove components at runtime

Disadvantages

❌ **Complexity**: High learning curve
❌ **Overhead**: Might be overkill for simple game
❌ **Indirection**: Harder to follow data flow
❌ **Boilerplate**: More code to set up
❌ **Debugging**: Behavior spread across systems

When to Use

- Large number of entities (1000+)

- Performance critical (AAA games)
- Need parallelization
- Complex entity interactions
- Building game engine framework
- Team experienced with ECS

ALTERNATIVE 4: Command Pattern with Memento

Design Overview

Commands as first-class objects, Memento for undo.

```java
```

```java
// Command interface
interface Command {
    void execute(GameContext context);
    void undo(GameContext context);
}

// Concrete commands
class MoveLeftCommand implements Command {
    private Position previousPosition;

    @Override
    public void execute(GameContext context) {
        Brick brick = context.getActiveBrick();
        previousPosition = brick.getPosition();
        brick.moveLeft();
    }

    @Override
    public void undo(GameContext context) {
        context.getActiveBrick().setPosition(previousPosition);
    }
}

class DropCommand implements Command {
    private List<Position> path = new ArrayList<>();

    @Override
    public void execute(GameContext context) {
        Brick brick = context.getActiveBrick();
        path.add(brick.getPosition());

        while (context.canMoveDown(brick)) {
            brick.moveDown();
            path.add(brick.getPosition());
        }
    }

    @Override
    public void undo(GameContext context) {
        context.getActiveBrick().setPosition(path.get(0));
    }
}

// Memento for state snapshots
class GameMemento {
    private final Field fieldSnapshot;
```

```java
    private final Brick brickSnapshot;
    private final int score;

    public GameMemento(GameContext context) {
        this.fieldSnapshot = context.getField().clone();
        this.brickSnapshot = context.getActiveBrick().clone();
        this.score = context.getScore();
    }

    public void restore(GameContext context) {
        context.setField(fieldSnapshot);
        context.setActiveBrick(brickSnapshot);
        context.setScore(score);
    }
}

// Command history
class CommandHistory {
    private Stack<Command> history = new Stack<>();
    private Stack<GameMemento> mementos = new Stack<>();

    public void execute(Command command, GameContext context) {
        mementos.push(new GameMemento(context));
        command.execute(context);
        history.push(command);
    }

    public void undo(GameContext context) {
        if (!history.isEmpty()) {
            Command command = history.pop();
            GameMemento memento = mementos.pop();
            memento.restore(context);
        }
    }
}
```

Advantages

- ✅ **Undo/Redo**: Built-in from the start
- ✅ **Macro Commands**: Combine commands
- ✅ **Command Queue**: Can delay/schedule commands
- ✅ **Replay**: Store and replay command sequences
- ✅ **Testing**: Easy to test individual commands
- ✅ **Logging**: Automatic action history

Disadvantages

❌ **Memory**: Storing mementos uses memory
❌ **Complexity**: More classes to manage
❌ **Coupling**: Commands need context access
❌ **Granularity**: Need right level of command size

When to Use

- Undo/redo is primary feature
- Need transaction-like behavior
- Recording/replaying gameplay
- Network play with command sync
- Tutorial system (scripted commands)

ALTERNATIVE 5: Reactive Programming (Rx/Flow)

Design Overview

Event streams and reactive operators.

```java
```

```java
// Using Reactor or RxJava
class ReactiveGame {
    private Flux<Command> commandStream;
    private Flux<GameState> stateStream;

    public ReactiveGame() {
        // Input stream
        commandStream = createCommandStream();

        // State stream derived from commands
        stateStream = commandStream
            .scan(GameState.initial(), (state, command) ->
                command.apply(state))
            .distinctUntilChanged()
            .share();

        // Auto-tick stream
        Flux.interval(Duration.ofMillis(500))
            .map(tick -> new TickCommand())
            .subscribe(commandStream::onNext);

        // Render subscription
        stateStream
            .subscribe(state -> render(state));

        // Match detection subscription
        stateStream
            .map(GameState::detectMatches)
            .filter(matches -> !matches.isEmpty())
            .subscribe(matches -> processMatches(matches));
    }

    private Flux<Command> createCommandStream() {
        return Flux.create(emitter -> {
            Scanner scanner = new Scanner(System.in);
            while (scanner.hasNext()) {
                String input = scanner.nextLine();
                Command cmd = parseCommand(input);
                emitter.next(cmd);
            }
        });
    }
}

// Commands as data
sealed interface Command {
```

```
    GameState apply(GameState state);
}

record MoveCommand(Direction dir) implements Command {
    @Override
    public GameState apply(GameState state) {
        return state.moveBrick(dir);
    }
}

record TickCommand() implements Command {
    @Override
    public GameState apply(GameState state) {
        return state.tick();
    }
}
```

Advantages

✅ **Declarative**: Describe data flow
✅ **Composable**: Chain operators easily
✅ **Async**: Natural async/concurrent handling
✅ **Backpressure**: Handle fast producers
✅ **Testing**: Marble testing for streams
✅ **Side Effects**: Explicit subscription points

Disadvantages

❌ **Learning Curve**: Reactive paradigm is complex
❌ **Debugging**: Stack traces are cryptic
❌ **Memory**: Stream chains allocate objects
❌ **Overkill**: Too much for simple game
❌ **Dependencies**: Need Rx library

When to Use

- Building real-time multiplayer
- UI with many async events
- Need sophisticated backpressure
- WebSocket/network game
- Team experienced with reactive
- Microservices architecture

ALTERNATIVE 6: State Machine (Explicit States)

Design Overview

Game as explicit state machine with transitions.

```java
```

```
// States
enum GamePhase {
    INITIALIZING,
    SPAWNING_BRICK,
    BRICK_FALLING,
    BRICK_LANDED,
    CHECKING_MATCHES,
    CLEARING_MATCHES,
    GAME_OVER
}

// State machine
class GameStateMachine {
    private GamePhase currentPhase = GamePhase.INITIALIZING;
    private GameData data;

    public void update() {
        GamePhase nextPhase = switch(currentPhase) {
            case INITIALIZING -> {
                initialize();
                yield GamePhase.SPAWNING_BRICK;
            }
            case SPAWNING_BRICK -> {
                if (canSpawnBrick()) {
                    spawnBrick();
                    yield GamePhase.BRICK_FALLING;
                } else {
                    yield GamePhase.GAME_OVER;
                }
            }
            case BRICK_FALLING -> {
                processInput();
                moveBrickDown();
                if (brickLanded()) {
                    yield GamePhase.BRICK_LANDED;
                }
                yield GamePhase.BRICK_FALLING;
            }
            case BRICK_LANDED -> {
                placeBrick();
                yield GamePhase.CHECKING_MATCHES;
            }
            case CHECKING_MATCHES -> {
                List<Match> matches = findMatches();
                if (!matches.isEmpty()) {
                    data.setMatches(matches);
```

```
                yield GamePhase.CLEARING_MATCHES;
            } else {
                yield GamePhase.SPAWNING_BRICK;
            }
        }
        case CLEARING_MATCHES -> {
            clearMatches();
            yield GamePhase.CHECKING_MATCHES;
        }
        case GAME_OVER -> {
            yield GamePhase.GAME_OVER;
        }
    };

    currentPhase = nextPhase;
  }
}
```

Advantages

✅ **Clarity**: Game flow is explicit
✅ **Debugging**: Easy to see current state
✅ **Validation**: Invalid transitions prevented
✅ **Testing**: Test each state independently
✅ **Visualization**: Can draw state diagram
✅ **Reasoning**: Clear about what can happen when

Disadvantages

❌ **Boilerplate**: Lots of switch statements
❌ **Rigidity**: Adding states requires changes everywhere
❌ **State Explosion**: Complex games have many states
❌ **Shared Data**: GameData accessed from all states

When to Use

- Game has clear phases
- Need explicit flow control
- Debugging state issues
- Team needs clarity on game flow
- Building turn-based game
- AI needs to reason about states

COMPARISON MATRIX

Approach	Complexity	Performance	Testability	Scalability	Learning Curve
Functional Immutable (Current)	Medium	Good	Excellent	Good	Medium
Event Sourcing	High	Medium	Excellent	Excellent	High
Mutable OOP	Low	Excellent	Medium	Medium	Low
ECS	High	Excellent	Good	Excellent	High
Command/Memento	Medium	Good	Good	Medium	Medium
Reactive	High	Good	Medium	Excellent	High
State Machine	Medium	Good	Excellent	Medium	Low

HYBRID APPROACHES (Best of Both Worlds)

Hybrid 1: Immutable State + Command Pattern

```java
// Immutable state (current approach)
// + Commands for undo/redo
interface Command {
    GameState execute(GameState state);
}

class CommandProcessor {
    private Stack<GameState> history = new Stack<>();

    public GameState execute(GameState state, Command command) {
        history.push(state);
        return command.execute(state);
    }

    public GameState undo() {
        return history.isEmpty() ? currentState : history.pop();
    }
}
```

Hybrid 2: ECS + Functional Operations

```java
// Components are immutable records
record Position(int row, int col) {}

// Systems return new world state
interface System {
    World update(World world);
}

// Immutable World
record World(Map<Entity, Position> positions,
             Map<Entity, Symbol> symbols) {

    public World updatePosition(Entity e, Position p) {
        Map<Entity, Position> newPos = new HashMap<>(positions);
        newPos.put(e, p);
        return new World(newPos, symbols);
    }
}
```

Hybrid 3: State Machine + Events

```java
// State machine controls flow
// Events for history/replay

class GameStateMachine {
    private GamePhase phase;
    private EventStore eventStore;

    public void transition(GamePhase next) {
        eventStore.append(new PhaseChanged(phase, next));
        this.phase = next;
    }
}
```

DECISION GUIDE: Which Approach to Choose?

Choose Functional Immutable (Current) When:

- ✅ Need thread safety

- ✅ Testability is priority
- ✅ Building distributed system
- ✅ Team knows functional programming
- ✅ **This is your BEST default choice**

Choose Event Sourcing When:

- ✅ Need complete audit trail
- ✅ Undo/redo is critical feature
- ✅ Building CQRS architecture
- ✅ Time-travel debugging required

Choose Mutable OOP When:

- ✅ Performance is proven bottleneck
- ✅ Single-threaded only
- ✅ Team is traditional Java
- ✅ Simplicity over robustness

Choose ECS When:

- ✅ 1000+ game entities
- ✅ Building game engine
- ✅ Need maximum performance
- ✅ Team experienced with ECS

Choose Command Pattern When:

- ✅ Undo/redo is main feature
- ✅ Need macro recording
- ✅ Transaction-like behavior
- ✅ Network command sync

Choose Reactive When:

- ✅ Heavy async/network I/O
- ✅ Real-time multiplayer
- ✅ Complex event streams
- ✅ Team knows reactive

Choose State Machine When:

- ✅ Clear game phases
- ✅ Turn-based game
- ✅ Need explicit flow
- ✅ AI reasoning about states

RECOMMENDATION FOR INTERVIEW

Best Answer Strategy:

1. **Start with your current approach**: "I chose functional immutable because it balances testability, clarity, and robustness."
2. **Show awareness of alternatives**: "For different requirements, I'd consider..."
    - Event Sourcing for audit trails
    - ECS for performance-critical games
    - State Machine for explicit flow control
3. **Demonstrate trade-off thinking**: "Each approach has trade-offs. Immutability trades memory for safety. ECS trades complexity for performance."
4. **Match to requirements**: "Given this is a console game with TDD focus, functional immutable is optimal. For a 3D game engine, I'd use ECS."
5. **Show you can adapt**: "I'm comfortable with all these patterns and would choose based on team expertise and project constraints."

This shows **depth**, **breadth**, and **pragmatism** - exactly what interviewers want!